

# GitHub Commands

A comprehensive reference guide to essential Git & GitHub CLI commands

## 1. Setup & Configuration

| Command                                               | Description                      |
|-------------------------------------------------------|----------------------------------|
| <code>git config --global user.name "Name"</code>     | Set your global username         |
| <code>git config --global user.email "e@x.com"</code> | Set your global email            |
| <code>git config --list</code>                        | List all configuration settings  |
| <code>git config --global core.editor "code"</code>   | Set VS Code as default editor    |
| <code>git config --global alias.st status</code>      | Create a shortcut alias (git st) |

## 2. Repository Setup

| Command                                        | Description                         |
|------------------------------------------------|-------------------------------------|
| <code>git init</code>                          | Initialize a new local repository   |
| <code>git init &lt;folder&gt;</code>           | Create a new repo in a named folder |
| <code>git clone &lt;url&gt;</code>             | Clone a remote repository locally   |
| <code>git clone &lt;url&gt; &lt;dir&gt;</code> | Clone into a specific directory     |
| <code>git clone --depth 1 &lt;url&gt;</code>   | Shallow clone (latest commit only)  |

## 3. Staging & Committing

| Command                              | Description                            |
|--------------------------------------|----------------------------------------|
| <code>git status</code>              | Show working tree status               |
| <code>git add &lt;file&gt;</code>    | Stage a specific file                  |
| <code>git add .</code>               | Stage all changes in current directory |
| <code>git add -p</code>              | Interactively stage chunks of changes  |
| <code>git commit -m "message"</code> | Commit staged changes with message     |
| <code>git commit --amend</code>      | Modify the most recent commit          |

| Command                                     | Description                          |
|---------------------------------------------|--------------------------------------|
| <code>git commit -am "message"</code>       | Stage tracked files and commit       |
| <code>git rm &lt;file&gt;</code>            | Remove a file and stage the deletion |
| <code>git mv &lt;old&gt; &lt;new&gt;</code> | Rename/move a file and stage change  |

## 4. Branching

| Command                                            | Description                        |
|----------------------------------------------------|------------------------------------|
| <code>git branch</code>                            | List all local branches            |
| <code>git branch -a</code>                         | List local and remote branches     |
| <code>git branch &lt;name&gt;</code>               | Create a new branch                |
| <code>git branch -d &lt;name&gt;</code>            | Delete a merged branch             |
| <code>git branch -D &lt;name&gt;</code>            | Force-delete a branch              |
| <code>git checkout &lt;branch&gt;</code>           | Switch to a branch                 |
| <code>git checkout -b &lt;branch&gt;</code>        | Create and switch to new branch    |
| <code>git switch &lt;branch&gt;</code>             | Switch to a branch (modern syntax) |
| <code>git switch -c &lt;branch&gt;</code>          | Create and switch (modern syntax)  |
| <code>git branch -m &lt;old&gt; &lt;new&gt;</code> | Rename a branch                    |

## 5. Merging & Rebasing

| Command                                        | Description                               |
|------------------------------------------------|-------------------------------------------|
| <code>git merge &lt;branch&gt;</code>          | Merge a branch into current branch        |
| <code>git merge --no-ff &lt;branch&gt;</code>  | Merge with a merge commit always          |
| <code>git merge --squash &lt;branch&gt;</code> | Squash all commits into one               |
| <code>git rebase &lt;branch&gt;</code>         | Rebase current branch onto another        |
| <code>git rebase -i HEAD~3</code>              | Interactive rebase for last 3 commits     |
| <code>git rebase --abort</code>                | Abort an in-progress rebase               |
| <code>git rebase --continue</code>             | Continue after resolving conflicts        |
| <code>git cherry-pick &lt;hash&gt;</code>      | Apply a specific commit to current branch |

## 6. Remote Repositories

| Command                                                | Description                            |
|--------------------------------------------------------|----------------------------------------|
| <code>git remote -v</code>                             | List remote connections                |
| <code>git remote add origin &lt;url&gt;</code>         | Add a remote named 'origin'            |
| <code>git remote rename &lt;old&gt; &lt;new&gt;</code> | Rename a remote                        |
| <code>git remote remove &lt;name&gt;</code>            | Remove a remote connection             |
| <code>git fetch</code>                                 | Download changes without merging       |
| <code>git fetch --all</code>                           | Fetch from all remotes                 |
| <code>git pull</code>                                  | Fetch and merge remote changes         |
| <code>git pull --rebase</code>                         | Fetch and rebase instead of merge      |
| <code>git push origin &lt;branch&gt;</code>            | Push branch to remote                  |
| <code>git push -u origin &lt;branch&gt;</code>         | Push and set upstream tracking         |
| <code>git push --force-with-lease</code>               | Force push (safe: checks remote state) |
| <code>git push origin --delete &lt;branch&gt;</code>   | Delete a remote branch                 |

## 7. Viewing History & Diffs

| Command                                                | Description                       |
|--------------------------------------------------------|-----------------------------------|
| <code>git log</code>                                   | Show commit history               |
| <code>git log --oneline</code>                         | Compact one-line commit history   |
| <code>git log --oneline --graph --all</code>           | Visual branch graph               |
| <code>git log -p</code>                                | Show history with patch diffs     |
| <code>git log --author="Name"</code>                   | Filter commits by author          |
| <code>git log --since="2 weeks ago"</code>             | Filter commits by date range      |
| <code>git diff</code>                                  | Show unstaged changes             |
| <code>git diff --staged</code>                         | Show staged changes               |
| <code>git diff &lt;branch1&gt;..&lt;branch2&gt;</code> | Compare two branches              |
| <code>git show &lt;hash&gt;</code>                     | Show details of a specific commit |
| <code>git blame &lt;file&gt;</code>                    | Show who changed each line        |

| Command                         | Description                 |
|---------------------------------|-----------------------------|
| <code>git shortlog -s -n</code> | Summarize commits by author |

## 8. Undoing Changes

| Command                                        | Description                            |
|------------------------------------------------|----------------------------------------|
| <code>git restore &lt;file&gt;</code>          | Discard unstaged changes in file       |
| <code>git restore --staged &lt;file&gt;</code> | Unstage a file (keep changes)          |
| <code>git revert &lt;hash&gt;</code>           | Create a new commit that undoes one    |
| <code>git reset HEAD~1</code>                  | Undo last commit, keep changes staged  |
| <code>git reset --soft HEAD~1</code>           | Undo last commit, keep changes         |
| <code>git reset --hard HEAD~1</code>           | Undo last commit, discard changes      |
| <code>git clean -fd</code>                     | Remove untracked files and directories |
| <code>git stash</code>                         | Stash current uncommitted changes      |
| <code>git stash pop</code>                     | Apply most recent stash and remove it  |
| <code>git stash list</code>                    | List all stashes                       |
| <code>git stash drop stash@{0}</code>          | Delete a specific stash                |

## 9. Tags & Releases

| Command                                           | Description              |
|---------------------------------------------------|--------------------------|
| <code>git tag</code>                              | List all tags            |
| <code>git tag &lt;name&gt;</code>                 | Create a lightweight tag |
| <code>git tag -a &lt;name&gt; -m "msg"</code>     | Create an annotated tag  |
| <code>git tag -a &lt;name&gt; &lt;hash&gt;</code> | Tag a specific commit    |
| <code>git push origin &lt;tagname&gt;</code>      | Push a tag to remote     |
| <code>git push origin --tags</code>               | Push all tags to remote  |
| <code>git tag -d &lt;name&gt;</code>              | Delete a local tag       |
| <code>git push origin --delete &lt;tag&gt;</code> | Delete a remote tag      |

## 10. Advanced & Useful Commands

| Command                                                   | Description                       |
|-----------------------------------------------------------|-----------------------------------|
| <code>git bisect start</code>                             | Start binary search for a bug     |
| <code>git bisect good &lt;hash&gt;</code>                 | Mark a commit as good             |
| <code>git bisect bad &lt;hash&gt;</code>                  | Mark a commit as bad              |
| <code>git reflog</code>                                   | Show history of HEAD movements    |
| <code>git submodule add &lt;url&gt;</code>                | Add a submodule to the repo       |
| <code>git archive --format=zip HEAD</code>                | Export repo as a zip archive      |
| <code>git grep "pattern"</code>                           | Search working tree for pattern   |
| <code>git worktree add &lt;path&gt; &lt;branch&gt;</code> | Check out branch in new directory |
| <code>.gitignore (file)</code>                            | List patterns of files to ignore  |

---

*Tip: Replace <placeholders> with your actual values. Run `git help <command>` for full documentation on any command.*